Report

This assignment implements a SmartParcel system using socket programming in Python. The system allows clients to register parcels, update their status, and look up parcel information through a TCP server, while also supporting a UDP-based health check.

Architecture Diagram

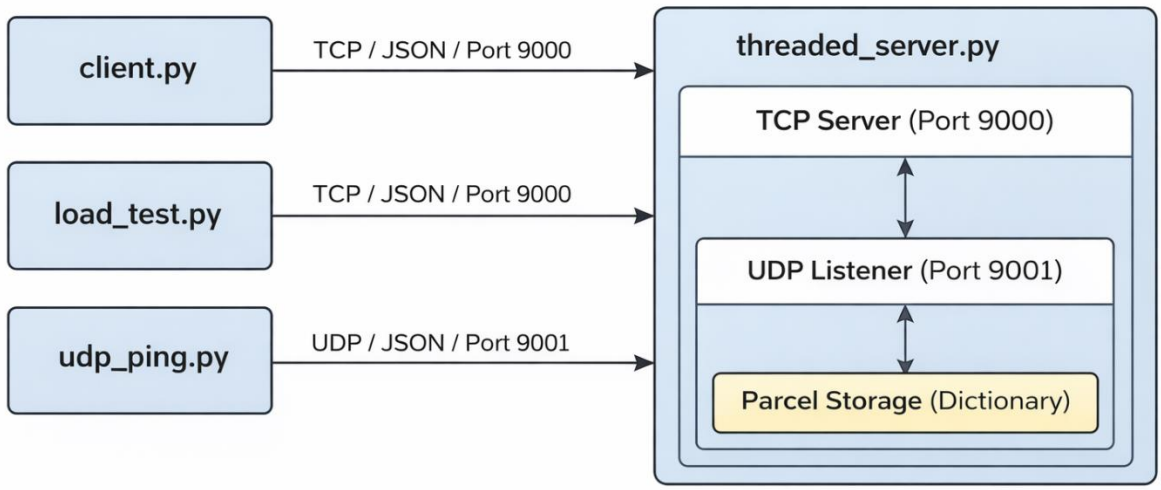


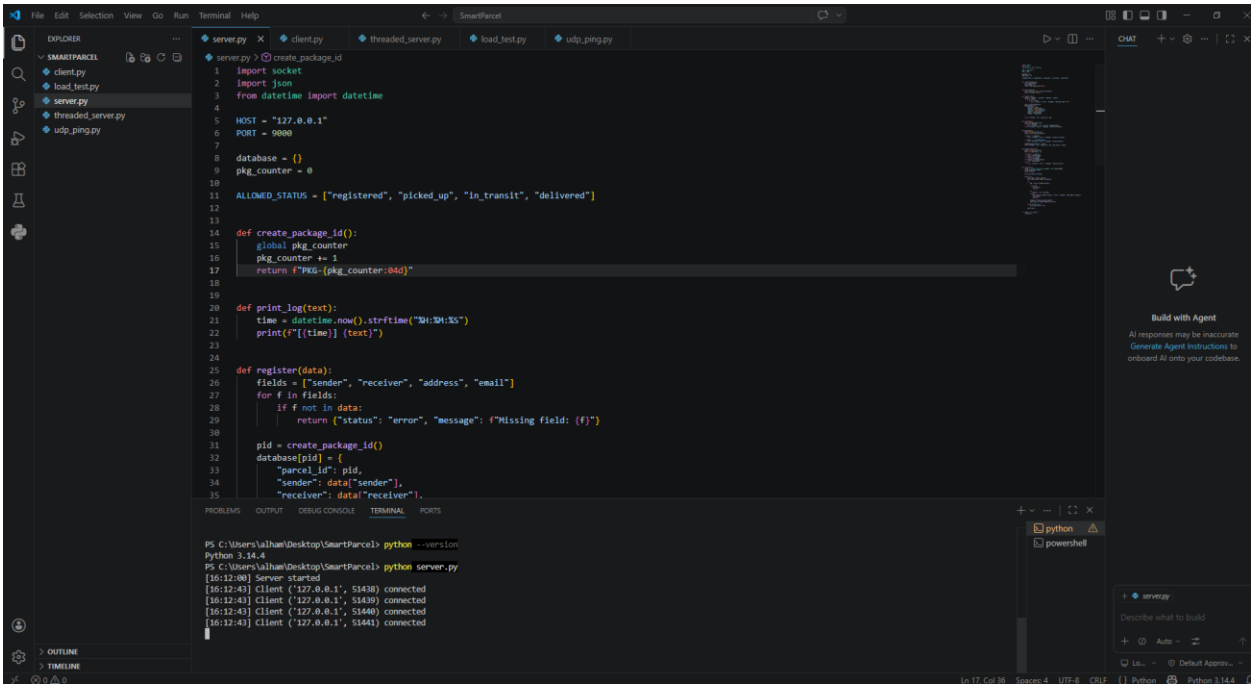
Figure: SmartParcel client-server architecture using TCP and UDP

This system follows a client-server architecture. The server (threaded_server.py) acts as the central system that listens for incoming requests and manages parcel data. Clients such as client.py, load_test.py, and udp_ping.py send requests to the server to perform operations.

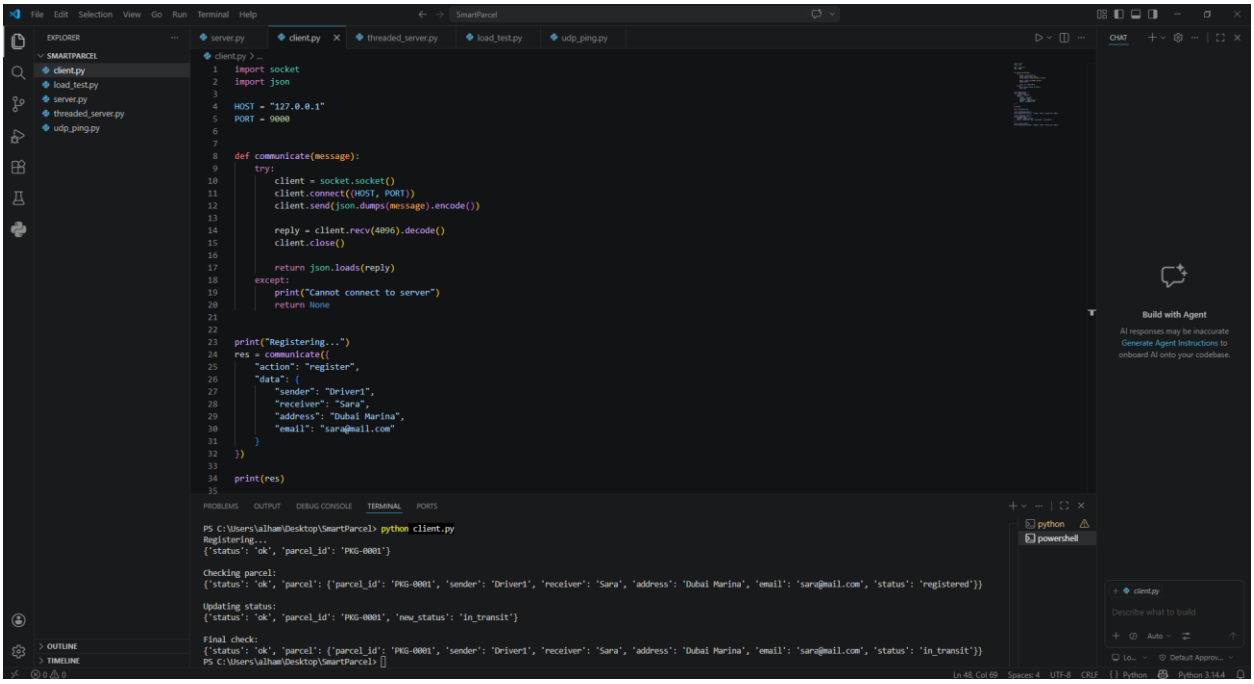
The server handles TCP connections on port 9000 for parcel operations such as registration, lookup, and updates. It also runs a UDP listener on port 9001 to respond to health check requests. Clients do not communicate with each other, only with the server, making this a client-server model rather than peer-to-peer.

Implementation

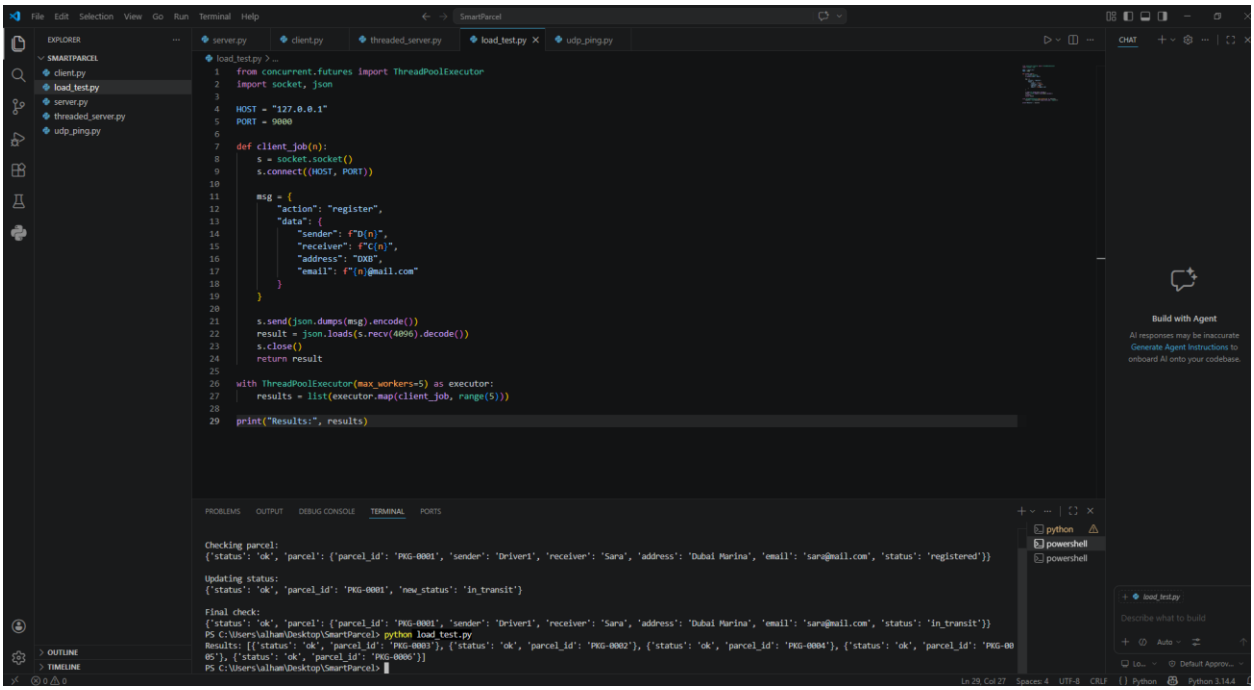
TCP Server Running



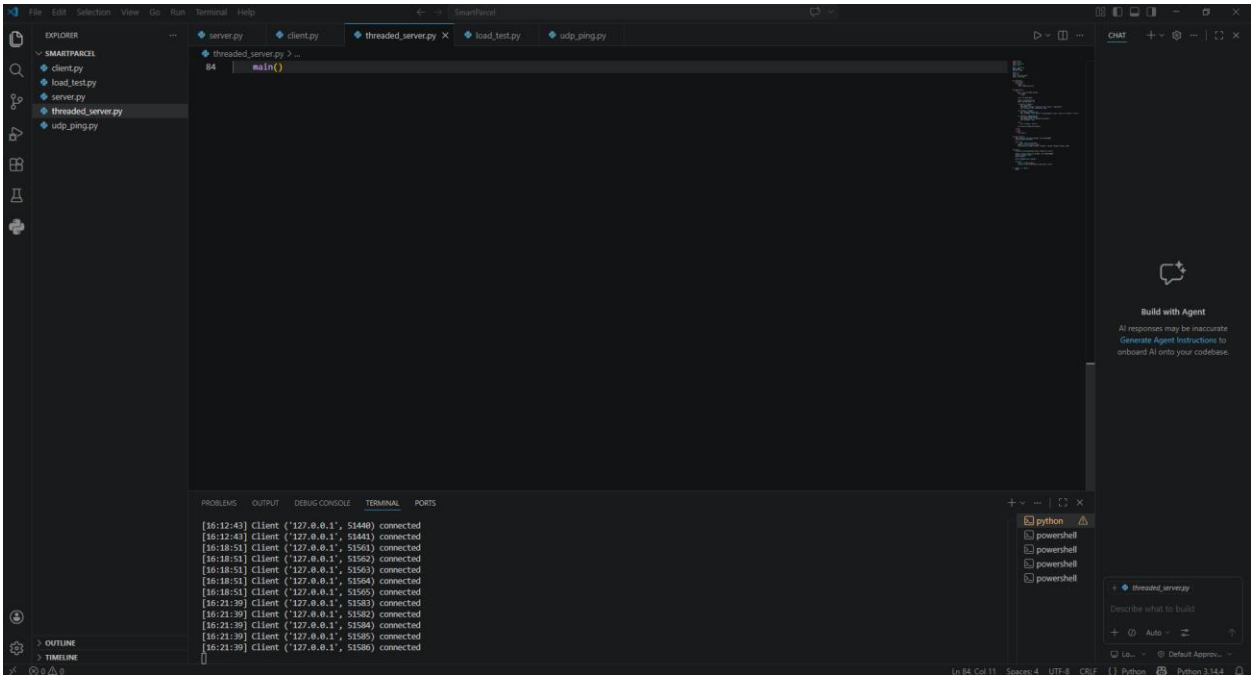
TCP Client Operations



Concurrent Client Load Test



Threaded Server Handling Multiple Clients



UDP Health Check

TCP vs UDP Explanation

UDP is suitable for a health check because it is fast and connectionless, meaning it does not require establishing a connection before sending data. Since health checks are simple and can be repeated if lost, reliability is not critical.

TCP is better for parcel registration because it provides reliable and ordered communication, ensuring that parcel data is delivered correctly without loss or duplication.

Conclusion

The SmartParcel system demonstrates how socket programming can be used to implement a client-server architecture. TCP ensures reliable communication for important operations such as parcel registration, while UDP provides a lightweight solution for quick health checks. The use of threading allows the server to handle multiple clients simultaneously, improving performance and scalability.